

Requirements Interaction Management in an eXtreme Programming Environment: A Case Study*

D.M. Woit
Ryerson University
Toronto, Ontario
Canada M5B2K3
1-416-979-5000

dwoit@scs.ryerson.ca

ABSTRACT

This work reports experience applying the process of Requirements Interaction Management (RIM) within the context of developing a commercial, internet-based software application in an industrial venue, employing the eXtreme Programming (XP) methodology. We describe our means of managing the interactions – by extending the standard XP requirements process while maintaining consistency with the principles of XP itself. We outline our rationale for modifying the XP requirements process, and provide a temporal comparison showing that, for this project, modified process is essential to successful application of RIM.

Categories and Subject Descriptors

D.2.1 [Software Engineering]: Requirements/Specifications – *methodologies, tools.*

General Terms

Measurement, Management, Documentation.

Keywords

requirements engineering, system specification, system analysis, dependency analysis, interaction analysis, extreme programming, user stories, requirements interaction management.

1. INTRODUCTION

1.1 The Product

Product development began in early 2000, using the standard eXtreme Programming (XP) methodology, as described in

[3,4,7]. The project was larger than any attempted at that time [7], comprising twelve developers, two graphic designers, two individuals filling the other XP roles (tracker, tester, coach, etc.), and up to three test individuals. The customer was on-site, devoted 100 percent of his time to the project, and provided specific and detailed GUI. The project was built for the customer, in accordance with his vision; upon completion, the product was to be sold or leased to third parties.

The product comprises three general utilities: a set of collaboration tools, a secure platform from which to build and deploy web services, and a system for distributed communication. The collaboration tools include a number of fundamental facilities, such as multimedia streaming, data sharing and storage, instant messaging, email, browsing, scheduling, etc., integrated through a distributed contact management system. The web services platform comprises an API supporting XML RPC and provides secure, third party integration with the system facilities mentioned above (scheduler, data storage, etc.) and associated data. The distributed communication system is a multi-domain, distributed server cluster that enables automated authentication and interaction within and between enterprises. Information is controlled by its owner, and is accessed by others, subject to authentication and permissions, via the cluster. The product is SSL secure, multi-language, and allows multi-branding. It has complex notions of client hierarchies, and all data sharing is regulated by user-manageable permissions, in accordance with the hierarchy. The multi-tasking and multi-dimensional GUI is complex (over 11,000 images) and dynamically re-configurable by both client and server. Although accessed via a browser, the product's interface with the Linux server is dynamic so that the browser DOM is reconfigured on the fly without user intervention.

1.2 The XP Requirements Process

The notion of requirements in XP differs from that of traditional methodologies in that user stories comprise requirements documents. A user story is typically 1-3 sentences written on an index card describing some functionality the customer values, serving as a starting point from which customer and developer generate more precise details [3,4]. Estimated completion time and notes are added to each card as it is discussed [10]. Developers decompose the functions on the card into manageable

*This work is supported by a Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grant and by Swurv Holdings Inc.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ICSE '05, May 15–21, 2005, St. Louis, Missouri, USA.
Copyright 2005 ACM 1-58113-963-2/05/0005...\$5.00

tasks, maintaining a record of each task and its status on the card. In the sequel, we will refer to the process of writing and using traditional story cards as the XP requirements process.

Modifications to the XP requirements process are reported necessary for projects with numerous story dependencies [1,17,18,20], projects that are large [5,12,20], and projects with a complex GUI component [19], all of which apply to our project. Our modifications to the requirements process are also essential to support management of requirements interactions as described in the sequel.

1.3 Requirements Interaction Management

Unmanaged dependencies among requirements have been attributed to stakeholder dissatisfaction and numerous software and system failures, some with grave consequences [8,15]. Requirements Interaction Management (RIM) is described as “the analysis and management of dependencies among requirements,” and is more formally defined as “the set of activities directed toward the discovery, management, and disposition of critical relationships among sets of requirements,” which “may be applied within an ad hoc or a defined process” [15]. We were motivated to attempt RIM incorporation because it is reported to be a “critical area of requirements engineering whose methods will lead to the development of systems with higher stakeholder satisfaction and fewer failures,” and because our project exhibits numerous dependencies (see Section 2) consistent with other projects reported to have benefited from RIM application [15]. We anticipated RIM to have a positive impact on the satisfaction of our XP customer, as predicted by [15]. We were motivated to modify the XP requirements process to incorporate RIM because application of RIM within the standard XP process proved temporally impractical, as explained in Section 3.

1.4 Outline

We begin, in Section 2, by confirming that the dependencies arising in our project requirements over a three-year period are consistent with those described in the literature. In Section 3 we describe our application of RIM, using the standard XP requirements process. We outline problems encountered and present our rationale for XP requirements process modifications in support of RIM. In Section 4 we describe our process modifications, which were developed in accordance with XP principles. Our application of RIM is outlined in Section 5. In Section 6 we present data showing RIM application temporally impractical before process modification, but practical afterward. We also include results concurring with [15] that RIM application results in higher stakeholder satisfaction. In Section 7 we outline alternative XP requirements process modifications and present our rationale for selecting our particular adaptations to accommodate RIM.

2. REQUIREMENTS INTERACTIONS

Data gathered over three years of our project show a total of 911 dependencies among 383 extant stories, with all dependencies originating in story cards. Forty percent of dependencies arise from functional interactions among stories. Sixty percent arise from GUI components, and are usually related to the customer’s explicit GUI artifacts included in stories.

Requirements dependencies are also referred to as requirements interactions [15] and are generally categorized as positive (the conjunction of the requirements does not imply False), negative (conjunction implies False), and implementation conflict (each requirement is satisfied individually by their implementations, but not when the implementations are combined.) All identified interactions in our project fall into the positive interaction category. No negative or implementation conflict interactions appear because they are effectively circumvented by our XP approach to RIM, described in Section 3, and our modifications to the XP requirements process, described in Section 4.

A “more refined analysis of the literature reveals that the basis of most interactions” can be classified as: structure, resource, task, causality and time [15], as follows:

- structure: two requirements are similar,
- resource: two requirements depend on the same resource,
- task: one requirement describes a task required for another,
- causality: one requirement describes a consequence of another, and
- time: temporal relations of one requirement to another, including coincident state.

Classification of our requirements interactions into these categories is presented in Table 1, revealing that the requirements dependencies we experienced are consistent with those reported in the literature [15].

Table 1. Classification into Literature Categories

Category	Percent of Interactions
Resource	63.8%
Causality	17.0%
Task	10.4%
Time	4.6%
Structure	4.2%

3. RATIONALE FOR PROCESS MODIFICATIONS

Managing requirements interactions follows the process: create the requirements, partition requirements to help identify interactions, resolve the interactions, and modify all relevant requirements [15]. The process may be performed in an ad hoc fashion, or employ specific tools, such as automated conflict analyzers applied to requirements recorded in a formal language. Requirements are analyzed for interactions typically after each phase in the lifecycle or after a major requirements modification. This appears to work well for projects developed with traditional methodologies. However, we chose to follow a more iterative approach, to be consistent with the principles of XP, including that of simplicity. As XP stories are developed one-by-one, usually extending current requirements or functionality, we chose to perform requirements interaction analysis, resolution, and modification during story creation. We found this more effective

in our environment because it allowed us to create stories with no negative interactions—as opposed to creating them with negative interactions and later analyzing the whole requirements set to discover them, followed by conflict resolution.

Consistent with the principles of XP, we initially employed the “simplest thing that could work” to identify and resolve requirements interactions during story creation [3]. We manually searched existing artifacts for dependencies, and if found, resolved them (or explicitly identified them if no resolution was necessary) and modified the appropriate artifacts immediately. However, identifying dependencies and modifying artifacts was challenging when we employed the traditional XP paper-and-pen method of artifact maintenance. The following activities were necessary, but tedious, time-consuming and error-prone:

- Searching of artifacts for related, existing functionality and GUI: This required visual inspection of great numbers of paper artifacts.
- Locating an artifact referenced from another: This required shuffling through large stacks of paper.
- Modification management of requirements artifacts. This required hand-written modifications to existing artifacts (natural language and diagrams), resulting in cognition and configuration problems.

We note that these activities may not be as challenging for every XP project. In our project, they are significant because of the extraordinary number of story artifacts and dependencies. Several underlying project properties contribute to the large number of artifacts and dependencies, as follows:

- Our customer has copious and exacting GUI requirements, down to the pixel, which he generates precisely using imaging software.
- Our product scope is much greater than the typical XP project [7,16].
- Our GUI is complex. It is dynamically generated based on system state and multiple user input simultaneously, resulting in a theoretically infinite number of configurations, which often require description in artifacts.

We expect any XP project with comparable properties will also encounter similar challenges incorporating RIM when employing the traditional XP requirements process. Other reports confirm that effective management of interactions requires process adaptations [1,17,19,20] for projects with extensive scope and GUI.

4. PROCESS MODIFICATIONS

We followed the basic tenants of XP in modifying our requirements process to facilitate RIM: We modified incrementally, incorporating user feedback; we modified process only as necessary to solve the problem at hand; we performed the simplest modifications that worked [3]. It is important to note that, in our case, the notion of “simple” is influenced by our collective experience with the Unix operating system (OS). The most important modifications were to make our artifacts electronically searchable and retrievable to facilitate interaction identification and requirements partitioning. We maintained

stories in text files that could be subject to the full arsenal of Unix searching utilities. The format chosen for text files was html because (a) hyperlinking allowed simple referencing of one artifact from another, useful for explicit dependency pointers, and (b) it is an OS-independent format, important because individuals in our project accessed requirements from Linux-, Windows- and Macintosh-based platforms. The customer created and modified stories with WYSIWYG software, while other individuals usually edited the html directly. The artifact revision process was also greatly simplified, being electronically based. Location of a named artifact now became trivial: artifacts were accessed through a browser by name, or through simple directory indexing; referenced artifacts were accessed via hyperlinks. Why we considered our strategy simpler than incorporating COTS tools is covered in Section 7.

As the requirements-base grew, feedback suggested adjustments as follows: It was desirable to replace flat indexing by a hierarchical organizational structure. A short CGI displays artifacts in a hierarchical fashion, sorted by component. As it was deemed undesirable to follow a number of links to retrieve an artifact, artifact links were listed sorted by sub-component, but presented flat. Our Macintosh-based customer usually located dependencies with the help of Unix-experienced members of the team; however, when he searched without such assistance, he was restricted to the searching capabilities of the browser. To facilitate this searching, we created a requirements view that concatenated artifact contents, so that all artifacts could be searched at once within the browser. An iteration view was added. This listed all of the XP iterations up to and including the current iteration, and within each, listed links of artifacts involved. Our simple solutions also applied to GUI artifacts. The GUI artifact creator, the customer, produced abundant requirements images by means of imaging software, which were easily referenced from other artifacts via linking. Finally, as the traditional XP requirements process requires space for notes on each story card, we accommodated this in our electronic version by adding a link to a notes file for each story.

4.1 Summary of Process Modifications

Our modifications to traditional XP process to accommodate RIM are summarized as follows:

- Replace paper-and-pen stories with electronic text documents in html format.
- Replace on-paper GUI drawings (hand-written, or created and printed with an imaging tool) with electronic copies, in any simple browser-compatible format, such as gif or jpg.
- Replace artifact searching via visual inspection with electronic searching.
- Replace artifact organization via paper files, stacks of paper, and hand-written lists, with an electronically based, hierarchical display.
- Replace artifact location via manual searching with electronic searching and retrieval through a hierarchical organizational structure.

5. APPLYING RIM

As each new story is created, it is subject to the standard RIM activities of requirements partitioning, interaction identification, interaction resolution, and requirements modification, as follows.

- Requirements partitioning: This is given at the most basic level by the requirements hierarchical structure. For example, email stories are isolated using glob constructs to identify them in the requirements hierarchy. Further partitioning is obtained via an ad hoc combination of Unix utilities. For example, to further partition to obtain a reverse-time ordered list of email stories created within the last week, but accessed yesterday, we would pipe find and sort utilities over files identified with glob constructs.
- Interaction identification: Dependencies are identified via efficient electronic searching at story creation time, using an ad hoc combination of Unix utilities, with grep being most commonly employed. Final identification involves reading targeted story fragments, lists or other information produced by grep partitioning.
- Interaction resolution: Once an interaction is identified, it is resolved by the customer. No conflict resolution methods as described in [15] are necessary, because only one customer contributes to requirements. Note that some interactions are identified for the purpose of adding an explicit dependency pointer to a story, and thus require no resolution. For example, when one story extends a permissions scheme developed in another story, the interaction is identified and a pointer to the original permissions scheme is included in the new story.
- Requirements modification: any required modifications are explicitly included in artifacts, by appropriate file editing.

The RIM activities normally result in additions/revisions to newly created stories. However, they can require revisions to existing stories, in the case when such stories are not yet implemented in the product. This typically occurs during the release-planning phase of XP, in which a group of stories is created at once for the next release.

6. EFFECTIVENESS OF PROCESS MODIFICATIONS

We assert that had we not modified our XP requirements process, we could not have successfully incorporated RIM, because of temporal drawbacks. Sampling the requirements artifact creation process showed that for each story created, an average of 3 complex interaction searches were required. Complex interaction searches are those that involve strategies beyond those of simple boolean textual location for the purpose of locating some functionality or image. They involved, for example, use of extended regular expressions; facilities to search by artifact properties, such as creation time, modification time, last access time, sub-component membership; summaries of number of times items were located within specific artifacts; sorting by various properties, etc. Data from pre-process

modification time shows that one such complex search averaged approximately 0.5 hours when performed manually, on approximately 25 stories. Extrapolating to a time in the project when approximately half the stories existed (about 150) suggests that had we been relegated to traditional XP requirements process for RIM, each new story would require $3 \times (0.5 \times 150 / 25) = 18$ hours in the worst case for complex searching. At best, the searches could be conducted in parallel, requiring only $(0.5 \times 150 / 25) = 6$ hours per new story. Sampling the modified requirements artifact creation process shows that each complex search requires an average of 5 minutes, attributed to search formulation and modification, and human cognition of the final computed results. Thus, for each new story the average complex search time is $3 \times 5 = 15$ minutes. We did not perform these in parallel because of hardware limitations.

Sampling the requirements artifact creation process showed that for each story created, an average of .5 artifact searches were required. An artifact search refers to simple manual locating of a given story card or image in the stack. Before process modification, this required a quick visual inspection of each artifact's name and/or classification, usually performed sequentially, as it was difficult to keep cards ordered. Data from pre-process modification time shows that one such artifact search averaged approximately 45 seconds when performed manually, on approximately 25 stories. Extrapolating to a time in the project when approximately half the stories existed (about 150) suggests that had we been relegated to traditional XP requirements process for RIM, each new story would require $0.5 \times (45 \times 150 / 25) / 60 = 2.25$ minutes for artifact searching. Sampling the modified requirements artifact creation process shows that each artifact search requires an average of 3 seconds, attributed to scrolling in a list to the correct area in the displayed hierarchy, visually locating the link and clicking it. Thus, for each new story the average artifact search time is $0.5 \times 3 = 1.5$ seconds.

Summing complex interaction search times and artifact search times before process modification gives 6 to 18 hours, plus 2.25 minutes per story. Summing these after modification gives 15 minutes plus 1.5 seconds. Therefore, incorporating RIM within the modified XP requirements process caused an additional 15 minutes (and 1.5 seconds) per new story—significant, but achievable. Had we not modified process, the project could not have succeeded, because 6-18 hours (plus 2.25 minutes) per new story is time prohibitive. The acute difference in time required for searches supports our assertion that RIM would be impossible to incorporate into our XP project without modifying the XP requirements process.

It is worth noting that our RIM application is consistent with higher customer satisfaction in the overall product, concurring with [15]. On a seven point Likert scale, the customer's satisfaction increased from point 2 (very dissatisfied) to point 6 (very satisfied) when measured pre-RIM and post-RIM incorporation, respectively. The modifications to the XP requirements process to accommodate RIM resulted in higher customer satisfaction with the XP requirements process itself. On a seven point Likert scale, the customer's satisfaction increased from point 1 (completely dissatisfied) to point 6 (very satisfied) when measured pre- and post-requirements modification.

We have presented data showing that for our project RIM incorporation is time-prohibitive unless the XP requirements process is modified, and that customer satisfaction increases with said process modifications. However, it is important to note that these are subject to this particular product, described in Section 1.1. Finally, based on the fact that our project was much larger than those suggested for the XP methodology [7], its size being in the top 4 percent of all XP projects [16], we believe that our successful application of RIM implies that our process modifications are appropriately scalable to any standard XP project.

7. COMPARISONS WITH OTHER PROCESS MODIFICATIONS

In [12] the authors provide suggestions for incorporating requirements engineering practices into XP, such as: electronic, web-based stories, customer GUI in gif or jpg format, hierarchical story organization, and searchable stories. The authors assert these practices improve the XP requirements process because they increase its Sommerville-Sawyer rating [12]. Our modifications to the XP requirements process are consistent with those of [12] and, in addition, are found to facilitate incorporation of RIM.

The need to alter the XP requirements process to effectively manage dependencies in large projects is noted in other reports [1,17,18,20], and our modifications provide specific solutions to the problems they identify. For another large project, [5], the standard XP requirements process is abandoned in favor of traditional up-front design. The solution for our large project was less extreme, requiring alteration of the standard XP requirements process only.

The traditional XP requirements process was modified to accommodate web-based projects [19]. However, those modifications are inadequate to aid in incorporating RIM, as they focus on assisting the customer in GUI production, which is unnecessary in our project, and do not make artifacts amenable to electronic searching and retrieval.

Others report modifying XP process to incorporate electronically based requirements artifacts, e.g., [2,3,9,11,21,22]. However, these reports focus on the tool support they supply, rather than on the process they modify. These tools are insufficient to effect the process modification required to meet the challenge of RIM in our project. None of the tools incorporate the complex searching facilities we required. None store artifacts in a format directly and independently accessible by more complex searching and retrieval utilities.

Tools for general requirements management (not XP specific) such as [6,8,14] allow some degree of complex searching, but not to the level we achieved with our simple Unix solution. These tools also impose a language structure on the requirements, which was unnecessary in our XP project and incompatible with the XP simplicity rule in our case.

Although the tools mentioned above are inadequate for our project because of their limited or non-existent searching and retrieval facilities, or their incompatibility with XP, they are also of limited applicability because of the following issues. These tools impose a fixed organizational structure on the

requirements. Tool users may not be able to easily adapt the structure to their own project. If tool modifications are possible, they may be challenging. In our project, incorporating user feedback about our requirements management “tool” was a simple task, involving modification of a small program that we created and maintained; however, adapting existing tools to incorporate our feedback, an obligation in an XP project, would have been an additional challenge if an open source tool had been used, and impossible with commercial tools. Flexibility, and ability to respond to user feedback, is thus limited.

It is also not clear how to incorporate the given tools into the existing revision control and backup facilities of our project. With our simple text files, revisions are easily maintained with cvs, consistent with the overall project. Backups require no special implementation, as they occur as part of the project backup process. With the tools above, artifacts are not necessarily stored in a form compatible with the revision and backup processes in place in our project. Enabling these facilities would require additional analysis and implementation, inconsistent with the simplicity rule.

Finally, many of the tools supporting electronic requirements are included in an overall XP CASE tool. Adoption of such a CASE tool necessitates the modification of more than simply the requirements process. In some projects, such as ours, this additional complexity and functionality is not required, as no feedback suggests that any non-requirements process modification would be beneficial. The basic principles of XP, directing us to modify only what is necessary now, and to do the simplest thing possible, suggest that adoption of a whole CASE tool, and its associated complexities, when most of its functionality is superfluous, is ill-advised.

8. CONCLUSIONS

We present our experiences applying Requirements Integration Management to our ongoing, in-production, commercial, eXtreme Programming project. We present our rationale for augmenting the standard XP requirements process to support RIM, and outline our adaptations. We use data gathered over three years to support our conclusion that these modifications are essential to successful RIM incorporation in our XP environment.

As discussed in the previous sections, we learned the following in relation to our large XP project with extensive, complex GUI requirements: RIM can be successfully incorporated using the principles of XP itself; incorporating RIM is not viable unless the standard XP requirements process is modified; complex, electronic requirements searching and retrieval are the most important accommodations to facilitate RIM; tool use is essential in modifying the XP requirements process to accommodate RIM, but use of COTS tools may not always be appropriate, and may be inconsistent with the principles of XP itself; our simple approach to XP requirements process modification is sufficiently scalable for standard XP projects; modifying the standard XP requirements process to accommodate RIM can result in increased customer satisfaction with the requirements process itself, and incorporating RIM can increase overall customer satisfaction with the product.

We hope that our experiences can help others in determining if their XP project is a candidate for RIM, and in determining if

process modifications are appropriate. Although each project and team must employ their own modifications in keeping with the principles of XP and in accordance with their own environment and experience, we believe our adaptations and lessons learned may be beneficial, and may serve as a guide, in their endeavors to successfully incorporate RIM into XP environments.

9. REFERENCES

- [1] Amr Elssamadisy, Gregory Schalliol. Recognizing and responding to "bad smells" in extreme programming. In *Proceedings of the 24th International Conference on Software Engineering* (Orlando FL, May 2002), 617-622.
- [2] Auer, K. Autotracker. In *Refining the practices of XP, workshop in ACM SIGPLAN Conference on object-oriented programming systems, languages and applications (OOPSLA 2002)* (Seattle, WA, Nov. 4-8, 2002).
- [3] Beck, K. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston MA, 2000.
- [4] Beck, K., and M. Fowler *Planning Extreme Programming*. Addison-Wesley, Boston, MA, 2001.
- [5] Cao, Lan, Mohan, K., Peng Xu, Ramesh, B. How extreme does extreme programming have to be? adapting XP practices to large-scale projects. In *Proceedings of the 37th Annual Hawaii International Conference on Software Engineering* (January 2004), 83-92.
- [6] DOORS Requirements Management System. <http://www.telelogic.com/>
- [7] eXtreme Programming Immersion Training. Feb 28 – March 3, 2000. Scotts Valley CA. Object Mentor.
- [8] Feather, M.S., Cornford, S.L, Gibbel, M. Scalable mechanisms for requirements interaction management. In *Proc. 4th Intl. Conference on Requirements Engineering* (Schaumburg, Illinois June 2000), 119-129.
- [9] Iterate. <http://www.diamond-sky.com/products/iterate/>
- [10] Jeffries, R., A. Anderson, and C. Hendrickson. *Extreme Programming Installed*. Addison-Wesley, Boston, MA, 2001.
- [11] Maurer, F., Supporting Distributed Extreme Programming. In *Lecture Notes in Computer Science*, Volume 2418/2002, Springer-Verlag Heidelberg, 2003, 13-22.
- [12] Nawrocki, J., Jasinski, M., Walter, B., Wojciechowski, A. Extreme programming modified: embrace requirements engineering practices. In *Proceedings IEEE Joint International Conference on Requirements Engineering* (Sept. 2002), 303-310.
- [13] Pinna, S, Mauri, S., Lorrai, P, and Corrigan, C. XPSwiki: An agile tool supporting the planning game. In *Lecture Notes in Computer Science, Volume 2675/2003*, Springer-Verlag Heidelberg, 2003, 104-113.
- [14] Rational RequisitePro <http://www-306.ibm.com/software/awdtools/reqpro>
- [15] Robinson, W.N., Pawlowski, S.D., and Volkov, V. Requirements interaction management. *ACM Computing Surveys* 35,2 (June 2003), 132-190.
- [16] Rumpe, B., Astrid Schroeder. Quantitative Survey on Extreme Programming Projects. In *Proceedings of International Conference on Extreme Programming and Flexible Processes in Software Engineerin, (XP2002)* (Alghero, Italy, May 2002) 95-100.
- [17] Schalliol, G. Challenges for Analysts on a Large XP Project. In *Extreme Programming Perspectives*. Ed. M. Marchesi, G. Succi, D. Wells, L. Williams. Addison-Wesley, Boston, MA, 2002.
- [18] Taber, C. and Fowler, M. An iteration in the life of an XP project. *Cutter IT Journal*, 13,11 (2000).
- [19] Wallace, D., I. Raggett, and F. Aufgang. *Extreme Programming for Web Projects*. Addison-Wesley, Boston, MA, 2003.
- [20] Watt, R.J., Leigh-Fellows, D. Acceptance test driven planning. In *Proceedings of the 4th Conference on Extreme Programming and Agile Methods* (Calgary, Canada, August 2004), 43-49.
- [21] Xpcgi. <http://xpcgi.sourceforge.net/>
- [22] Xplanner. www.xplanner.org/